

Methodology

1.1 Introduction

Transformer encoders such as BERT remain attractive for text classification because they offer strong accuracy, stable training behaviour, and simple deployment interfaces. Their practical weakness is that a standard finetuned encoder is often much larger than the downstream task requires. The dense model spends compute on every attention head and every feed forward neuron at every layer, even when a large fraction of those structures can be removed without destroying the task solution. This report studies how to choose those structures under a strict inference budget and how to recover the accuracy of the resulting sparse architecture without retraining a new model from scratch.

The compression setting considered here is deliberately structured. The method removes complete attention heads and complete feed forward neurons, and in some cases removes an entire transformer layer by making it an identity map. This is different from unstructured weight pruning, where individual scalar weights are zeroed. The reason is operational rather than aesthetic. A dense smaller matrix is easy to export, run, profile, and compare; an irregular sparse matrix requires runtime support that may not exist on the deployment target and often fails to deliver a proportional speedup. The goal of the report is therefore not merely to reduce the number of non zero parameters, but to construct architectures whose forward pass is genuinely cheaper in the dimensions that matter for inference.

The central difficulty is that the architecture selection problem is combinatorial. A BERT-base encoder has twelve layers, twelve heads per layer, and 3072 feed forward neurons per layer. A candidate architecture is a binary decision over all of these structures. Exhaustive enumeration is impossible, and direct evaluation of a candidate by fully training or fully recovering it is too expensive to use inside a search loop. The method therefore separates the problem into two phases. The first phase searches over binary architectures using a cheap calibration metric. The second phase takes the selected architecture and performs a multi-stage recovery procedure based on distillation from the finetuned teacher.

The search phase is not a continuous relaxation of the masks. It is a direct search over hard binary masks. This choice avoids the gap between a relaxed gate and the binary architecture eventually deployed, but it also means that the search needs a robust sampling procedure. The report uses an evolutionary algorithm with a hard compute budget. The budget is expressed as a fixed operation count at a reference sequence length, and candidates are repaired into a narrow feasibility band around that budget. This constraint is essential: without it, a teacher-matching surrogate naturally favours keeping more structure, and the search degenerates into a size comparison rather than an allocation comparison. Inside the budget band, candidates have similar cost and differ mainly in where compute is placed across depth, attention, and feed forward capacity.

The surrogate metric used during search compares the masked student and the dense teacher on a calibration subset. It uses representation and prediction signals that are cheap enough to compute for many candidates. The report treats this metric as a practical instrument, not as an oracle. Several experiments show that the metric can mis-rank architectures, especially at high sparsity, because the unadapted masked student is not the same object as the recovered student. For that reason, the search includes diversity preservation, candidate caching, multi-fidelity

evaluation, and a final funnel rather than trusting a single scalar score blindly.

The recovery phase is equally important. A selected mask can have very low raw accuracy before adaptation. The recovery procedure first hardens the masks and establishes the masked model state, then uses intermediate representation distillation to rebuild the internal trajectory of the student, and finally uses prediction distillation and task loss to improve the classifier-facing behaviour. The distinction between these stages matters empirically. On QNLI, for example, the representation recovery stage accounts for most of the accuracy restoration, while later prediction finetuning provides a smaller final correction. The report therefore describes the recovery procedure in detail rather than treating it as a generic finetuning step.

The contribution of the work is the full system: a hard-budget structured search over heads and neurons, a calibration metric designed for fast candidate evaluation, an evolutionary search procedure with diversity and caching machinery, and a multi-stage recovery method that makes the selected architecture usable. The experimental sections evaluate the resulting method on GLUE classification tasks, compare against simpler search baselines, ablate the search and recovery components, and diagnose where the metric succeeds or fails as a predictor of post-recovery accuracy.

The remainder of this chapter is organised as follows. Sections 1.2 through 1.5 define the candidate representation, the cost model, and the calibration signals. Sections 1.4 through 1.7 describe the surrogate metric and the search algorithm. Section 1.8 describes the recovery procedure. The final sections report the experimental protocol and conclusions.

1.2 Mask Parameterisation

This section defines the architecture variable optimised by the search. Each candidate is represented by two binary masks: one over attention heads and one over feed forward neurons. The masks do not change the embedding dimension, classifier interface, residual topology, or layer ordering. They only decide which structured units remain active, matching the unit level used by structured transformer pruning methods (Michel et al., 2019; Xia et al., 2022; Kurtic et al., 2023).

1.2.1 Candidate Space

A candidate architecture is a pair of binary masks

$$c = (\mathbf{M}^H, \mathbf{M}^N) \in \{0, 1\}^{L \times H} \times \{0, 1\}^{L \times N}. \quad (1.1)$$

For layer ℓ , the active attention heads and active feed forward neurons are

$$H_\ell(c) = \{h : \mathbf{M}^H[\ell, h] = 1\}, \quad N_\ell(c) = \{n : \mathbf{M}^N[\ell, n] = 1\}. \quad (1.2)$$

The head mask selects complete head slices in the attention projections. The neuron mask selects rows of the first feed forward projection and the matching columns of the second feed forward projection. The masks are indexed in the original teacher coordinates.

The remaining definitions are derived from this same object: structural counts, layer status, and candidate identity.

1.2.2 Structural Counts

The first derived object is the count vector induced by the two masks. The count vector summarizes the mask for budget control and reporting. At layer ℓ , summing the binary decisions

in each row gives

$$a_\ell(c) = \sum_{h=1}^H \mathbf{M}^H[\ell, h], \quad (1.3)$$

$$n_\ell(c) = \sum_{n=1}^N \mathbf{M}^N[\ell, n]. \quad (1.4)$$

Thus $a_\ell(c)$ records how many attention heads remain in layer ℓ , whereas $n_\ell(c)$ records how many feed forward neurons remain in that same layer. The search keeps these local quantities because cost and behaviour depend on where units are kept.

The global head and neuron counts are then obtained by summing the local counts across depth:

$$A(c) = \sum_{\ell=1}^L a_\ell(c), \quad N(c) = \sum_{\ell=1}^L n_\ell(c). \quad (1.5)$$

These totals report candidate size. They do not show whether the remaining units are concentrated in a few layers or distributed across many layers. Therefore, we also track the number of layers that contain at least one active sublayer:

$$L_{\text{act}}(c) = \sum_{\ell=1}^L \mathbf{1}\{a_\ell(c) + n_\ell(c) > 0\}. \quad (1.6)$$

Finally, the same per-layer counts are used to evaluate the compute budget. Each active head contributes one head cost, and each active feed forward neuron contributes one neuron cost:

$$\text{MAC}(c) = \sum_{\ell=1}^L (a_\ell(c)c_H + n_\ell(c)c_N), \quad (1.7)$$

where c_H and c_N are the per head and per neuron costs derived in Section 1.3.

The pair (a_ℓ, n_ℓ) also fixes the execution case of layer ℓ . Table 1.1 shows how these counts determine the masked forward pass. The search therefore selects both the budget and the computation that remains at each depth.

Condition	Status	Masked forward
$a_\ell > 0, n_\ell > 0$	both active	attention and feed forward execute
$a_\ell = 0, n_\ell > 0$	feed forward only	attention update is zero; feed forward executes
$a_\ell > 0, n_\ell = 0$	attention only	attention executes; feed forward update is zero
$a_\ell = 0, n_\ell = 0$	identity	the layer returns its input

Table 1.1: Layer statuses induced by the structural masks.

This status is used whenever a candidate is evaluated. Positive counts keep the corresponding sublayer active, whereas zero counts remove that sublayer update. Therefore, two candidates with the same global size can behave differently if their active units create different layer statuses. The next step is to define how the two mask types interact inside an active layer.

1.2.3 Why Heads and Neurons Are a Joint Choice

The search represents $\mathbf{M}^H$ and $\mathbf{M}^N$ as a single candidate. A head decision changes the residual stream before the feed forward block, and a neuron decision changes how that stream is

transformed. Let x_ℓ be the input to layer ℓ . With active head set H_ℓ and active neuron set N_ℓ , the masked layer can be written abstractly as

$$u_\ell = \text{LN}_\ell^A(x_\ell + A_\ell(x_\ell; H_\ell)), \quad (1.8)$$

$$x_{\ell+1} = \text{LN}_\ell^F(u_\ell + F_\ell(u_\ell; N_\ell)). \quad (1.9)$$

Here A_ℓ is the attention update restricted to the surviving heads, and F_ℓ is the feed forward update restricted to the surviving neurons. Because F_ℓ is evaluated at u_ℓ , changing H_ℓ changes the input seen by the surviving feed forward neurons. A mutation to one mask can therefore change the effect of the units selected by the other mask. The search scores, caches, and mutates the pair jointly.

After this coupling is fixed, the implementation needs two levels of candidate description. The exact description prevents repeated evaluations of the same mask. The coarser description tracks the shape of the population.

1.2.4 Granularity and Candidate Identity

Neuron choices are grouped for search efficiency. The implementation allocates feed forward neurons in blocks of sixteen. Thus $n_\ell(c)$ is restricted to multiples of sixteen, except for the zero case. This block size keeps mutation aligned with meaningful cost changes and reduces candidates that differ only by negligible single-neuron moves.

For exact caching, the identity of a candidate is the bit string obtained by concatenating $\mathbf{M}^H$ and $\mathbf{M}^N$. Two candidates are identical if and only if this bit string is identical. This key is deliberately fine-grained because it preserves the exact placement of every active unit. For diversity tracking, the search also records the coarser macro signature

$$\sigma(c) = (A(c), N(c), L_{\text{act}}(c)). \quad (1.10)$$

The macro signature loses placement information. Two candidates can have the same total counts and active layer count while assigning compute to different depths, so exact deduplication still uses the full bit string. The macro signature is used only as a population-level descriptor in the island and diversity mechanisms of Section 1.6.2.

1.3 Efficiency Budget and Cost Model

The mask defined in Section 1.2 deactivates a subset of attention heads and feed-forward neurons in each layer. Since the active heads and neurons are fixed by the mask, the computational cost of each candidate is determined directly by that mask. The remaining components form a smaller sub-network whose forward pass evaluates fewer active parameters than the dense model. To compare candidates under a fixed compute target, we express this cost as a MAC count derived from the mask. This value is used consistently for feasibility checks during search, repair of infeasible offspring, and final reporting. All costs are measured at the reference sequence length S used for calibration, so every candidate is compared under the same input shape.

The following subsections define this MAC count from the active attention heads and feed-forward neurons that remain after pruning. We first compute the cost of one active head and one active feed-forward neuron, then aggregate these costs within each layer and across the encoder.

1.3.1 Per Unit Costs

The unit costs follow directly from the linear projections that survive under the mask. For one active attention head with head dimension d_h , the input and output projections give the first order term

$$c_H = 2Sdd_h, \quad (1.11)$$

where the contributions that do not depend on the head mask are absorbed into a constant and dropped. For one active feed forward neuron the two linear maps give

$$c_N = 2Sd, \quad (1.12)$$

the first selecting the neuron from the residual stream and the second projecting it back. Both unit costs are independent of layer index, so the cost of a layer is determined by the number of active heads and active neurons it contains.

1.3.2 Candidate Cost

Let $a_\ell(c)$ and $n_\ell(c)$ denote the active head and neuron counts of layer ℓ under the mask of candidate c . The layer cost is the linear combination

$$\text{MAC}_\ell(c) = a_\ell(c)c_H + n_\ell(c)c_N, \quad (1.13)$$

and the candidate cost is the sum across depth,

$$\text{MAC}(c) = \sum_{\ell=1}^L \text{MAC}_\ell(c). \quad (1.14)$$

The dense reference $\text{MAC}_{\text{dense}} = L(Hc_H + Nc_N)$ gives the cost of the unmasked model, while $\rho(c) = \text{MAC}(c)/\text{MAC}_{\text{dense}}$ reports each candidate as a fraction of dense cost. This global value does not replace the layer-wise decomposition, this implies that the global cost alone is not sufficient: two masks can have equal cost while distributing compute across different layers. The surrogate of Section 1.4 therefore depends on the placement of compute, not only on its total amount.

1.3.3 Example Decomposition

We instantiate the cost model on BERT-base, with $L = 12$, $H = 12$, $d = 768$, $d_h = 64$, $N = 3072$, and reference length $S = 128$. Substituting into (1.11) and (1.12) gives the unit costs; the layer and total costs of the illustrative candidate of Table 1.2 follow from (1.13) and (1.14).

Table 1.2: Per layer decomposition. Unit costs from (1.11)–(1.12):
Costs in M ops.

ℓ	a_ℓ	n_ℓ	Head	Neur	Tot
0	2	112	25	22	47
1	3	96	38	19	57
2	2	64	25	13	38
3	2	64	25	13	38
4	0	16	0	3	3
5	0	16	0	3	3
6	0	0	0	0	0
7	1	32	13	6	19
8	0	32	0	6	6
9	1	32	13	6	19
10	0	16	0	3	3
11	1	16	13	3	16
Σ	12	496	152	97	249

$$c_H = 2 \cdot 128 \cdot 768 \cdot 64 \approx 12.58 \text{ M},$$

$$c_N = 2 \cdot 128 \cdot 768 \approx 0.197 \text{ M}.$$

Layer 0 ($a_\ell = 2$, $n_\ell = 112$):

$$\text{MAC}_0 = 2c_H + 112c_N \approx 47.2 \text{ M}.$$

Candidate total and dense reference:

$$\text{MAC}(c) \approx 249 \text{ M}, \quad \text{MAC}_{\text{dense}} \approx 9060 \text{ M}.$$

Keep ratio:

$$\rho(c) = 249/9060 \approx 0.027.$$

The candidate retains roughly 2.7% of dense cost; compute is concentrated in the lower layers and the middle is nearly emptied.

1.3.4 Feasibility Band

The target budget B is not generally reachable exactly. Head counts are integers and neuron counts are allocated in blocks of sixteen, so the set of attainable values of $\text{MAC}(c)$ is discrete and the chance that any single point coincides with B is small. The search therefore accepts a candidate as feasible whenever its cost lies in a narrow band around the target,

$$(1 - \varepsilon)B \leq \text{MAC}(c) \leq (1 + \varepsilon)B, \quad (1.15)$$

which keeps candidates at a comparable compute level and reduces the comparison between them to the question of where the budget is placed, not how much of it is spent. The half width ε is a hyperparameter of the search, chosen small enough that the band does not confound compute reductions with metric improvements and large enough that repair operators have a non empty target set. Operation count is used for the constraint because it is deterministic and device independent; a deployment specific latency table can replace it without affecting the rest of the procedure.

The budget model therefore fixes the size of the candidate and leaves the allocation problem open. Several masks can satisfy the same operation band while placing their heads and neurons at very different depths. The next section defines the surrogate metric used to choose among these equal-cost candidates before any recovery training is performed.

1.4 The Surrogate Distance Metric

After budget filtering, the candidates being scored are already close in operation count. The remaining question is allocation. The same budget can be placed at different depths and split differently across the two structural axes. A useful search metric therefore has to compare how these allocations behave, since the cost alone no longer separates them.

The metric is evaluated before recovery training. This restriction is essential because the search scores thousands of masks, and running a full distillation or fine tuning procedure for every candidate would make the search infeasible. The metric therefore has to judge an unrecovered masked model from a small calibration pack. Validation accuracy and final task loss are weak by themselves at this stage because high-sparsity candidates often produce flat or collapsed outputs.

The implementation uses two ranking steps. The first step ranks candidates by residual-update preservation and keeps the best fraction of the cohort. The second step ranks the survivors by a task-facing distance. This order reflects the role of the metric in the search: a candidate must first preserve a plausible computation path, then the task signal decides which preserved path is already closest to the teacher’s prediction surface.

1.4.1 Equal-Cost Candidate Ranking

The budget constraint turns a large architecture space into a local comparison problem. The metric receives a cohort of feasible allocations near the target cost, so the candidates differ mainly in where their remaining units are placed.

Let $\mathcal{C}_t$ be the cohort evaluated at generation t . Every $c \in \mathcal{C}_t$ satisfies the feasibility band

$$(1 - \varepsilon)B \leq \text{MAC}(c) \leq (1 + \varepsilon)B. \quad (1.16)$$

The band makes the members of $\mathcal{C}_t$ comparable in cost. Small residual MAC differences inside the band are treated as secondary; the main comparison is the placement of the retained heads and feed forward neurons. This is why the score is cohort-local: it decides which feasible allocation should survive this generation.

For each candidate, the teacher $\mathcal{T}$ and masked student $\mathcal{S}(c)$ are evaluated on a calibration pack

$$\mathcal{D}_{\text{cal}} = \{x_i\}_{i=1}^B.$$

Let $p_{i,s} \in \{0, 1\}$ indicate whether token position s of example i is non-padding. All hidden-state and update distances are computed only over positions where $p_{i,s} = 1$.

The output of the metric is a cohort-local ordering. Lower values are better:

$$\Phi(c_1) < \Phi(c_2) \implies c_1 \text{ is preferred to } c_2 \text{ inside the current search generation.}$$

This ordering drives search-time selection before recovery. The first quantity needed by the two-step rule is the structural rank, so the metric starts from the layer updates.

1.4.2 Residual Update Matching

The structural rank is based on a simple intuition: a retained architecture should reproduce the transformations made along the teacher’s depth. Comparing final hidden states is too coarse for this purpose. A hidden state contains the accumulated residual stream, so a weak layer can be hidden by computation that happened earlier. The layer update exposes the contribution made at one depth.

Let h_ℓ denote the hidden state after layer ℓ , with h_0 the embedding output. The layer update is

$$\Delta h_\ell = h_\ell - h_{\ell-1}. \quad (1.17)$$

For the teacher and student, these updates are written as $\Delta_\ell^{(\mathcal{T})}$ and $\Delta_\ell^{(\mathcal{S})}$.

The update Δh_ℓ isolates the computation performed at layer ℓ . If a masked layer contributes almost nothing, then $\Delta_\ell^{(\mathcal{S})}$ becomes small even when the absolute hidden state remains numerically close to the teacher stream. This makes update matching a better search signal for partial and fully dropped layers.

For tensors A and B shaped as batch, sequence, and hidden dimension, define the calibration inner product

$$\langle A, B \rangle_{\text{cal}} = \sum_{i=1}^B \sum_{s=1}^S p_{i,s} A_{i,s}^\top B_{i,s}, \quad \|A\|_{\text{cal}} = \sqrt{\langle A, A \rangle_{\text{cal}}}. \quad (1.18)$$

The inner product sums only real token positions, so padding does not change the distance. The per-layer trajectory distance then checks two properties of the student update. Direction agreement asks whether the student update points the same way as the teacher update. Projection agreement asks whether the student recovers the teacher update with the right scale along that direction:

$$d_{\ell, \text{cos}}^{\text{traj}}(c) = \frac{1}{\pi} \arccos \left(\frac{\langle \Delta_\ell^{(\mathcal{S})}, \Delta_\ell^{(\mathcal{T})} \rangle_{\text{cal}}}{\|\Delta_\ell^{(\mathcal{S})}\|_{\text{cal}} \|\Delta_\ell^{(\mathcal{T})}\|_{\text{cal}} + \delta} \right), \quad (1.19)$$

$$\alpha_\ell(c) = \frac{\langle \Delta_\ell^{(\mathcal{S})}, \Delta_\ell^{(\mathcal{T})} \rangle_{\text{cal}}}{\|\Delta_\ell^{(\mathcal{T})}\|_{\text{cal}}^2 + \delta}, \quad (1.20)$$

$$d_{\ell, \text{proj}}^{\text{traj}}(c) = |1 - \alpha_\ell(c)|, \quad (1.21)$$

$$d_\ell^{\text{traj}}(c) = (1 - \lambda) d_{\ell, \text{cos}}^{\text{traj}}(c) + \lambda d_{\ell, \text{proj}}^{\text{traj}}(c). \quad (1.22)$$

The constant δ is a numerical stabiliser. The cosine term measures direction agreement between the student and teacher updates. The projection coefficient α_ℓ measures how much of the teacher update is recovered by the student along the teacher direction. A value near one means the student update has the correct magnitude in the teacher direction. A value near zero means the update is missing or orthogonal. Values above one mean the student overshoots the teacher update.

Both parts are needed. A layer with the right norm and wrong direction fails the cosine term. A layer with the right direction and weak magnitude fails the projection term. A dropped layer gives $\Delta_\ell^{(S)} \approx 0$, so its projection coefficient is close to zero. The per-layer distance therefore penalises missing, misdirected, and over-scaled computation.

1.4.3 Aggregation Across Layers

A candidate is selected as a whole architecture, so the layer distances need a single structural score. The aggregation below turns the sequence of per-layer errors into one trajectory distance:

$$d^{\text{traj}}(c) = \frac{1}{Z} \sum_{\ell=1}^L w_\ell d_\ell^{\text{traj}}(c), \quad Z = \sum_{\ell=1}^L w_\ell. \quad (1.23)$$

The weights w_ℓ determine how depth contributes to the structural score. The simplest choice is $w_\ell = 1$, which gives each layer equal influence. A magnitude-weighted variant uses a function of $\|\Delta_\ell^{(T)}\|_{\text{cal}}$, so layers where the teacher update is larger receive more weight. A late-layer-weighted variant can emphasise depths closer to the classifier. These choices change the search pressure, so the experiments report them as metric ablations.

The trajectory score is the first-stage rejector in the metric. Candidates with many missing, misdirected, or scale-wrong updates receive high d^{traj} . Candidates that preserve a coherent computation path receive lower d^{traj} . At this point the metric has identified which masks still perform a plausible internal computation; the next stage can then use final-output evidence to order those masks.

1.4.4 Task-Facing Distance After Structural Filtering

The structural score does not inspect the final label decision. The second ranking stage fills that gap with task-facing evidence from the final logits. This signal is useful after the update gate, because the surviving candidates have already preserved enough internal computation to make their logits meaningful.

Let $z_i^{(T)}$ and $z_i^{(S)}(c)$ be the teacher and student logits for input x_i . The distillation distance is

$$d^{\text{kd}}(c) = T^2 \frac{1}{B} \sum_{i=1}^B \text{KL} \left(\text{softmax}(z_i^{(T)}/T) \parallel \text{softmax}(z_i^{(S)}(c)/T) \right), \quad (1.24)$$

where T is the distillation temperature. The hard task-facing term is

$$d^{\text{ce}}(c) = -\frac{1}{B} \sum_{i=1}^B \log \text{softmax}(z_i^{(S)}(c))_{y_i^*}, \quad (1.25)$$

where y_i^* is either the true label or the teacher argmax label, depending on the calibration mode.

At high sparsity, many unrecovered candidates produce logits that are nearly uniform, biased toward one class, or unstable under small mask changes. In that regime, d^{kd} and d^{ce} can be similar for masks with different internal computations. This is the reason the search does not use the task distance as the first gate. The task distance is written as

$$d^{\text{task}}(c) = \beta_{\text{kd}} d^{\text{kd}}(c) + \beta_{\text{ce}} d^{\text{ce}}(c), \quad (1.26)$$

with non-negative coefficients. This distance is computed for every candidate in the cohort, then converted to a percentile rank. The final selection rule uses this task rank after the update-rank gate has selected the structurally acceptable candidates.

A low task distance from a collapsed candidate can occur for accidental reasons on a small calibration pack. A low task distance from a candidate with good residual updates is more meaningful, since the candidate has preserved a computation path capable of supporting recovery.

1.4.5 Two-Step Cohort Ranking

The search implementation uses the double-rank gate in `updt_gate` mode. This is the point where the two distances become the actual selection score. Raw trajectory and task distances have different units, ranges, and noise levels, so the implementation compares candidates by percentile ranks within the current cohort.

For any component d , define the lower-is-better percentile rank

$$r_d(c) = \frac{1}{\max(|\mathcal{C}_t| - 1, 1)} \text{rank}_{\mathcal{C}_t}(d(c)), \quad (1.27)$$

where ties receive their averaged rank and lower values are better. The two ranks used by the search are

$$r^{\text{traj}}(c) = r_{d^{\text{traj}}}(c), \quad r^{\text{task}}(c) = r_{d^{\text{task}}}(c).$$

Here r^{traj} comes from the trajectory distance and r^{task} comes from the task-facing distance. The ranking rule then uses them in a fixed order.

The first ranking step keeps candidates whose update rank is inside the retained fraction ρ :

$$\mathcal{S}_t = \{c \in \mathcal{C}_t : r^{\text{traj}}(c) \leq \rho\}, \quad \rho = \text{UPDT_GATE_KEEP_FRAC}.$$

This step implements the structural filter: candidates outside $\mathcal{S}_t$ have weaker update preservation than the retained fraction of the cohort. The second step orders the survivors by task rank, with a small update-rank tie-break:

$$\Phi(c) = \begin{cases} r^{\text{task}}(c) + \epsilon r^{\text{traj}}(c), & c \in \mathcal{S}_t, \\ 1 + r^{\text{traj}}(c), & c \notin \mathcal{S}_t. \end{cases} \quad (1.28)$$

The first branch is the survivor ordering used for selection. The second branch assigns every rejected candidate a score above one, placing it behind the survivor set. Rejected candidates still keep their update-rank order, so the genetic search retains pressure toward better structure even among candidates that fail the current gate.

1.4.6 Calibration Noise and Resampling

The two-step ranking depends on measurements from a finite calibration pack. A candidate near the update gate can change rank if the pack changes. The implementation reduces this variance by averaging component distances over independent packs before computing ranks. Let $\mathcal{P}_1, \dots, \mathcal{P}_R$ be independently sampled calibration packs. A resampled component is computed as

$$\bar{d}(c) = \frac{1}{R} \sum_{r=1}^R d_{\mathcal{P}_r}(c). \quad (1.29)$$

The same averaging applies to d^{traj} , d^{kd} , and d^{ce} before cohort ranks are computed.

The search uses two fidelity levels. Cheap evaluation scores many candidates on a small pack and drives broad exploration. Full evaluation averages over larger or multiple packs for candidates that may survive, enter the Hall of Fame, or be compared across generations. The cheap score gives speed. The full score reduces rank flips near the structural gate.

Resampling is especially important around κ . A small calibration perturbation can move a candidate from survivor to non-survivor. Averaging reduces these boundary errors and makes the selected population less dependent on one calibration draw.

1.4.7 Limits of the Surrogate

The surrogate ranks unrecovered candidates. It cannot guarantee recovered validation accuracy. Recovery can repair a candidate with a mediocre initial surrogate score, and recovery can fail on a candidate whose initial updates look good. The metric is therefore evaluated empirically by rank correlation with post-recovery validation accuracy in the later diagnostic experiments.

1.5 Calibration Data and Signal Collection

The search metric in Section 1.4 is evaluated many times, so it cannot use the full training or validation set for every candidate. It uses a small calibration subset instead. This subset supplies the inputs on which the teacher and the masked student are compared during search.

1.5.1 Calibration Subset

The calibration subset is sampled from the training split with a fixed seed. The validation split is reserved for reporting recovered accuracy, so it is not used as the search signal. This keeps the search target separate from the final evaluation target.

Each calibration example is tokenised in the same way as the task data used for training and recovery. Padding positions are tracked, because hidden-state and update distances are computed only on real tokens. Labels are kept for the task-facing terms of the surrogate, such as cross entropy, but the calibration subset is not used to update model parameters during search.

1.5.2 Cached Teacher Signals

The teacher is fixed while candidate masks are being searched. Its outputs on the calibration subset are therefore computed once and reused. The cached signals are the teacher logits and the teacher hidden states at each layer.

The logits support the task-facing part of the metric. The hidden states support the residual-update distance, since layer updates are computed from consecutive hidden states. Caching these teacher signals removes the teacher forward pass from each candidate evaluation. Candidate scoring then requires forwarding only the masked student on the same calibration inputs and comparing its outputs to the cached teacher signals.

1.5.3 Cheap and Full Packs

The implementation uses two calibration fidelities. A cheap pack contains fewer calibration tokens and is used when many candidates must be screened quickly. A full pack contains more tokens and is used for periodic full evaluations, Hall-of-Fame candidates, and final reranking.

Both packs are built from the same calibration subset. The cheap pack gives the search enough signal to discard weak candidates at low cost. The full pack reduces noise when a candidate is important enough to affect the population or the final selected architecture.

For full evaluation, the implementation can average over multiple independently sampled full packs. This resampling is used before cohort ranks are computed, so the update-rank gate and the task-rank ordering depend on averaged component distances rather than on a single calibration draw.

1.6 The Genetic Search Algorithm

The search stage receives the fixed mask representation, the compute budget, the calibration packs, and the surrogate metric defined in the preceding sections. Its role is to select one feasible architecture for recovery. The search cannot use gradients with respect to the binary masks,

and exhaustive enumeration is infeasible. The procedure therefore uses an evolutionary search that keeps multiple feasible candidates alive, evaluates only a bounded number of masks, and returns the best mask found under the surrogate ranking.

1.6.1 Inputs and Search Objective

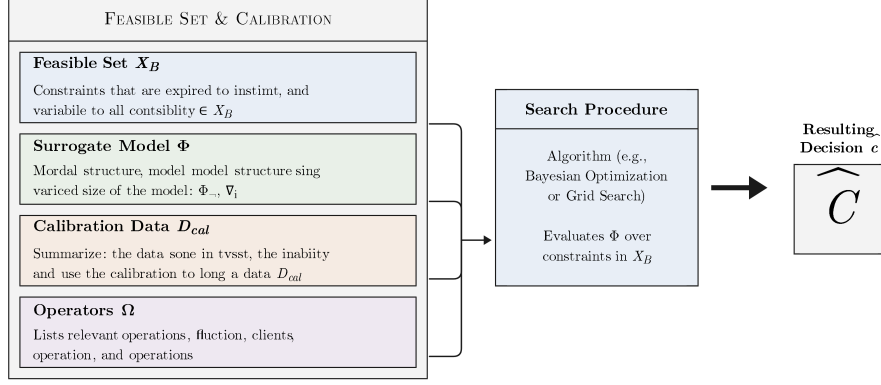


Figure 1: The main inputs are in feasible between the feasible Set X_B in λ . The Surrogate Model Φ is struggled-line constraints and operation analysis and relevant constraints in Ω .

Figure 2: Search Procedure Algorithm. c . Evaluates Φ over constraints in X_B .

Final Resulting Decision $\hat{c}$

Figure 1.1: Inputs and output of the search. The four objects fixed by Sections 1.2–1.4 are passed to the search procedure, which returns a single mask $\hat{c}$.

Figure 1.1 shows the contract of the search. A candidate $c = (\mathbf{M}^H, \{n_\ell\}_{\ell=1}^L)$ specifies the surviving heads and feed-forward neurons. The feasible set is the hard budget band

$$\mathcal{X}_B = \{c : (1 - \varepsilon)B \leq \text{MAC}(c) \leq (1 + \varepsilon)B\}.$$

Each candidate is scored on the calibration packs $\mathcal{D}_{\text{cal}}^{\text{cheap}}$ and $\mathcal{D}_{\text{cal}}^{\text{full}}$ by the surrogate distance Φ . The algorithm also enforces executable layer statuses and the active-depth floor $N_a(c) \geq L_{\min}$.

The search objective is the inexact discrete optimisation problem

$$\hat{c} \in \arg \min_{c \in \mathcal{X}_B} \Phi(c). \quad (1.30)$$

Equation 1.30 defines the target of the search, not a guarantee of global optimality. The method spends a finite evaluation budget to find a strong candidate in a large combinatorial space. This input contract leads directly to the five-phase evolutionary procedure below.

1.6.2 The Five-Phase Genetic Procedure

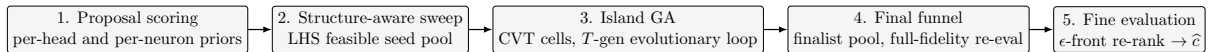


Figure 1.2: Five-phase overview of the genetic search. Phase 1 computes proposal scores from the teacher on the calibration set. Phase 2 builds a feasible seed pool by a structure-aware sweep. Phase 3 runs the island GA: a CVT partition of $\mathcal{X}_B$ and a T -generation evolutionary loop with archive updates. Phase 4 pools finalists and re-evaluates them at full fidelity. Phase 5 selects $\hat{c}$ from the pooled finalists.

Phase 1: Proposal Scoring

Before any candidate is constructed, the procedure computes per-head and per-neuron Fisher-style local sensitivities of the teacher on the calibration set, denoted $s_H[\ell, h]$ and $s_N[\ell, n]$. The per-layer aggregate

$$\text{imp}(\ell) = \sum_h s_H[\ell, h] + \sum_n s_N[\ell, n]$$

serves as a layer importance prior. These quantities are not the search objective; the surrogate Φ of Section 1.4 remains the objective. The proposal scores serve three roles: they decide which heads and neurons each operator adds or removes, they order block-level additions and removals during repair, and they define the ranking of layers from which the protected core during repair is drawn.

Phase 2: Structure-Aware Sweep

The initial seed pool is built by a Latin hypercube sweep over three orthogonal axes: the active-layer count N_a , the fraction ϕ_H of the operation budget spent on attention heads, and a subset mode that decides how the active layers are drawn from $\text{imp}(\ell)$. For each cell of the LHS grid, a candidate is constructed with the chosen N_a active layers and the chosen split of MAC between heads and neurons, then repaired into $\mathcal{X}_B$. The sweep returns a pool of feasible candidates spread across coarse macro descriptors. This pool seeds the Phase-3 populations and provides the sample from which the CVT partition of Phase 3 is fitted.

Phase 3: Island GA

The third phase exploits a two-scale structure of $\mathcal{X}_B$. Feasible neighbours obtained by a single budget-neutral swap differ by a small structural edit, while widely separated masks differ qualitatively at the same MAC count by placing capacity at different depths or by changing the head–neuron balance. The first regime calls for local refinement; the second calls for exploration across distinct regions.

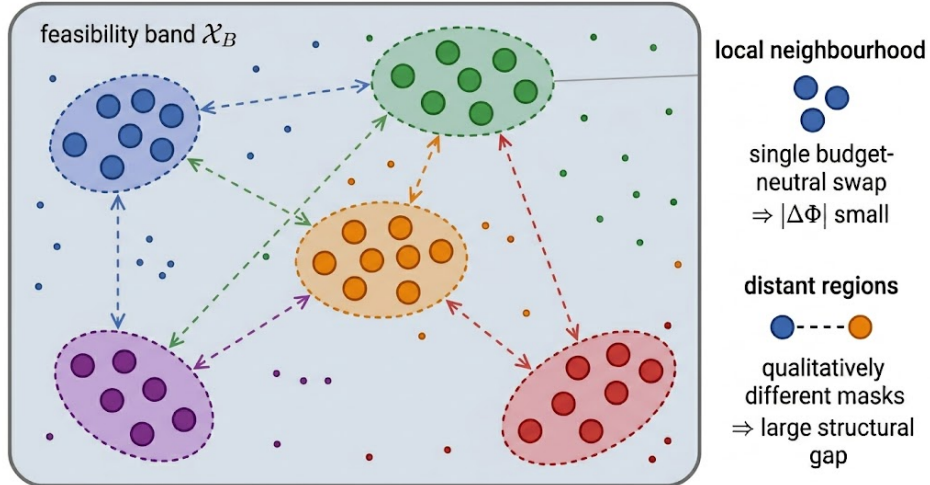


Figure 1.3: The feasibility band $\mathcal{X}_B$ as a populated space. Feasible candidates form clusters of architecturally similar masks; clusters in different regions of the band differ qualitatively.

The third phase encodes this two-scale structure by partitioning $\mathcal{X}_B$ into Voronoi cells via k -means on a six-coordinate architectural descriptor $z(c) \in [0, 1]^6$ fitted to the sweep pool. The coordinates capture the share of MAC spent on heads, the neuron fraction, the active-layer fraction, the entropy of the per-layer budget distribution, and the centres of mass of heads and

of neurons along the depth axis. Each cell defines an island’s basin; a population of size P is seeded inside each basin from the sweep pool of Phase 1.6.2, projected into the basin’s bounds. Local refinement is then carried out inside each island, while the partition itself enforces coverage across regions.

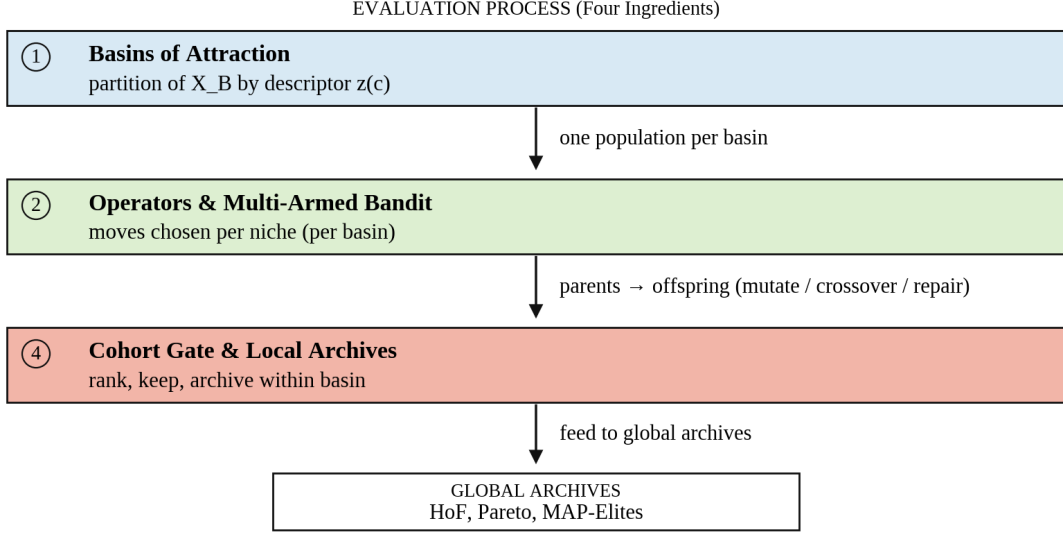


Figure 1.4: The four pieces of machinery inside the generational loop. Regions maintain separate populations, operators produce feasible offspring after repair, bandits adapt operator probabilities, and selection feeds both the next population and the archives.

Within this partition, the loop runs for T generations. In each generation, per island: parents are chosen by a tournament rule whose score combines a fitness-shared rank (each rank divided by a structural-crowding count) with a novelty bonus equal to the minimum mask-distance to already-selected parents; offspring are produced by crossover and mutation operators drawn from an EXP3-IX bandit local to the island; each offspring is projected back into the basin to enforce the budget band and the basin bounds; offspring are evaluated by Φ at cheap fidelity, with a scheduled subset re-evaluated at full fidelity; and the union of incumbents and offspring is ranked by the cohort gate of Section 1.4. Survivors form the next-generation population. The best-scoring candidates of the cohort are inserted into three global archives: a Hall of Fame keyed by exact mask signature, a Pareto front on (Φ, MAC) , and a MAP-Elites grid keyed by (N_a, ϕ_H) .

Phase 4: Final Funnel

After T generations, the procedure assembles a finalist pool from four sources: the global best candidate observed across the run, the top candidates of each island’s final population, samples from the three archives (Hall of Fame, Pareto front, MAP-Elites), and the global best running incumbent. The pool is deduplicated by exact mask signature. Each finalist is then re-evaluated on the full-fidelity calibration pack with R independent resamples averaged into a single score.

Phase 5: Fine Evaluation

The averaged finalist scores define an ϵ -front: the candidates whose resampled Φ lies within ϵ of the lowest score in the finalist pool. The ϵ -front is sorted lexicographically by $(\Phi, -\text{MAC})$, so among candidates within calibration noise of one another the procedure prefers the one that uses more of the allowed compute. The first element of the sort is returned as $\hat{c}$.

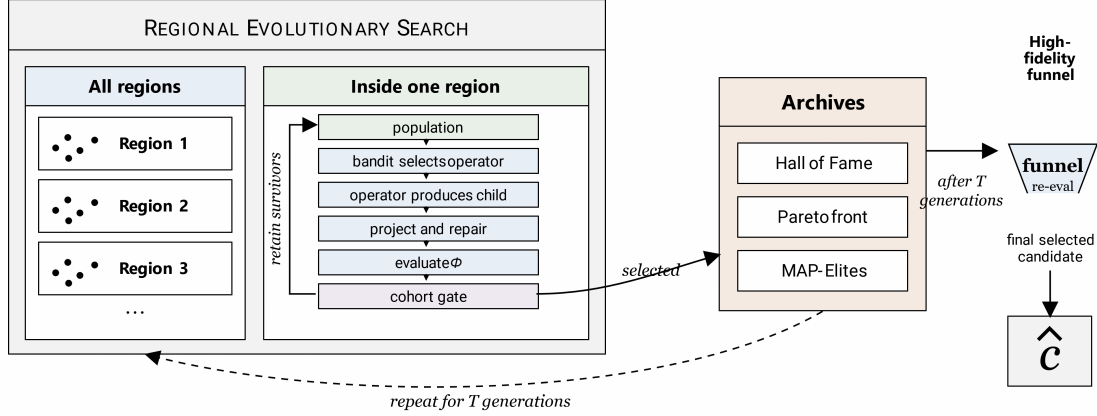


Figure 1.5: One iteration of the search and the final funnel. Each region’s population produces offspring with operators chosen by its bandit; offspring are projected back into the region, evaluated on the calibration packs, ranked by the cohort gate, and used to update the population and the archives. After T iterations, the archives and the per-region best candidates are re-evaluated at higher fidelity and one mask $\hat{C}$ is returned.

1.7 Alternative Search Baselines

The evolutionary search of Section 1.6 is a complex object, and a reader is entitled to ask whether the complexity is necessary. This section describes two baseline search procedures that operate on the same surrogate metric, the same calibration set, and the same compute budget as the evolutionary search. The purpose of the baselines is to isolate the contribution of the evolutionary machinery from the contribution of the surrogate metric itself.

1.7.1 Baseline One: Random Search

The first baseline draws a fixed number of candidates uniformly from the feasible set and returns the candidate with the lowest surrogate score on the full calibration pack.

The sampling procedure works as follows. A random Latin hypercube configuration is drawn, in the same form as the bootstrap of Section 1.6, namely an active layer count, a head fraction, and a subset mode. The configuration is converted to a candidate by the importance based allocation procedure, and the candidate is repaired into the feasibility band. The result is added to the pool of evaluated candidates. The procedure repeats until the total number of forward passes matches the compute budget allocated to the evolutionary search.

The random search uses the same calibration packs and the same surrogate metric as the evolutionary search. The two procedures therefore differ only in how candidates are proposed.

The random search is the strongest possible baseline against the claim that the evolutionary machinery adds value over the metric. If the random search achieves comparable recovered accuracy at comparable compute, the evolutionary machinery is superfluous. If the random search achieves lower recovered accuracy, the difference is the value added by the evolutionary machinery.

1.7.2 Baseline Two: Beam Search

The second baseline replaces the population structure of the evolutionary search with a beam of fixed width and replaces the bandit operator selection with a uniform sampling of operators.

The procedure works as follows. A beam of B candidates is initialised by the Latin hypercube bootstrap. At each iteration, each candidate in the beam is expanded by applying a fixed number

of mutation operators sampled uniformly at random from the operator set of Section 1.6.2. The resulting offspring are scored on the cheap surrogate, and the top B candidates across the union of the previous beam and the new offspring are retained. The procedure repeats for a fixed number of iterations.

The beam search is intermediate in complexity between the random search and the evolutionary search. It uses the same operators as the evolutionary search but lacks the island structure, the bandit, and the cohort relative ranking.

1.7.3 Fairness Constraints

The comparison of the three procedures is meaningful only if they operate under identical resource budgets. We enforce four constraints.

First, the total number of full forward passes is the same across the three procedures, equal to the product of the evolutionary search’s per generation population size and number of generations.

Second, the calibration set is fixed across the three procedures, including the cheap and full packs and the resamples.

Third, the budget constraint and the feasibility band are the same.

Fourth, the post search recovery procedure is the same. The chosen architecture of each baseline is passed through the multi phase recovery of the search sectionsI with identical hyperparameters, and the validation accuracy of the recovered model is reported.

1.7.4 What the Comparison Shows

The comparison is reported with the experimental results. We summarise here the qualitative role of each baseline.

The random search achieves a lower recovered accuracy than the evolutionary search at the same compute budget. The gap widens at higher sparsity. The reason is that the feasible set at high sparsity contains a small fraction of architectures with non degenerate recoverability, and random sampling reaches that small fraction with probability proportional to its measure. The evolutionary search reaches the small fraction faster because the bandit and the cohort selection bias the search toward architectures whose local moves improve the surrogate.

The beam search achieves a recovered accuracy intermediate between the random search and the evolutionary search. The beam search inherits the locality of the operators but does not benefit from the diversity preservation of the island structure or from the operator weighting of the bandit. Its main failure mode is premature convergence onto a narrow region of the macro signature space.

The qualitative pattern is consistent with the methodological claim that the evolutionary machinery is not redundant on top of the surrogate metric, and that the search procedure contributes a measurable fraction of the methodology’s empirical advantage.

1.7.5 Why Not Compare Against Continuous Relaxation Methods

A reader may ask why we do not include a continuous relaxation method as a baseline, for example a method that trains the masks as continuous variables under a sparsity regulariser. The reason is that such a method is not a search at all in the sense used in this report. It is a parameter training procedure with mask training built in, and the recovered model is the output of the same training that produces the masks.

The comparison of a continuous relaxation method against the methodology described here is meaningful but it would require equalising training compute across the two methods, which is a substantially more involved experimental design than the head to head search baselines of this section. We treat the comparison as out of scope and refer the reader to the published

comparisons in Xia et al. (2022) and Kurtic et al. (2023) for the relative merits of the two paradigms.

1.8 Recovery Process

The search returns a fixed pruned architecture $\hat{c} = (\mathbf{M}^H, \mathbf{M}^N)$. At high sparsity, this architecture usually requires parameter recovery before evaluation: the surviving heads and neurons inherit the teacher parameters, whereas the residual pathway is altered by the removed components. Recovery therefore trains the selected masked student while keeping the architecture fixed.

The recovery objective combines representation-level and prediction-level supervision. These objectives impose distinct constraints on the compressed model. Hidden-state matching aligns the internal trajectory with the teacher computation, whereas label and logit supervision align the final predictive distribution. Under severe compression, the available capacity may be insufficient to optimise both objectives effectively from the initial step. The recovery procedure is therefore staged. The first stage uses intermediate representation matching to stabilise the surviving layers. The second stage shifts the optimisation target to prediction-level distillation and task supervision. This follows the general principle of knowledge distillation (Hinton et al., 2015) and the use of intermediate supervision in compact language models (Jiao et al., 2020).

Figure 1.3 summarises this recovery logic. It places the fixed mask before all optimisation steps, then separates internal representation recovery from final prediction recovery.

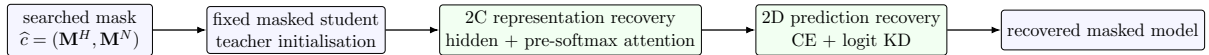


Figure 1.6: Recovery flow after architecture search. The mask is fixed throughout recovery; only the surviving parameters are trained.

The left-to-right order in Figure 1.3 fixes the implementation order used by the recovery process. The student parameters are initialised from the teacher. The head and neuron masks are hardened to binary tensors and are not optimised. Fully dropped layers are bypassed as identities during masked training. The classifier is frozen during the representation stage and unfrozen for prediction recovery. The procedure therefore separates architecture selection from parameter recovery: search chooses the structure, and the recovery process trains the selected structure.

1.8.1 2C: Intermediate Representation Recovery

The first recovery stage trains the masked student before the classifier is allowed to adapt. Its purpose is to restore an internal trajectory that can support subsequent prediction training. If prediction loss is applied immediately, classifier gradients are propagated through representations that may already be poorly aligned with the teacher. Hidden-state supervision gives the surviving layers a direct target and supplies a denser signal than the task label alone.

Let the teacher hidden states be

$$h_0^{(\mathcal{T})}(x), h_1^{(\mathcal{T})}(x), \dots, h_L^{(\mathcal{T})}(x),$$

where $h_0^{(\mathcal{T})}$ is the embedding output. Let $\mathcal{S}(\hat{c})$ be the ordered set of active student layers under the selected mask. A fixed set of teacher anchor depths $\mathcal{A}$ is chosen along the teacher depth. The mapping

$$\pi : \mathcal{S}(\hat{c}) \rightarrow \mathcal{A}$$

assigns each active student layer to a teacher anchor according to its relative depth. This mapping is dynamic because the active student layers depend on the searched architecture. A shallow candidate and a deeper candidate therefore use different anchor assignments.

For a batch $\mathcal{B}$ and a padding mask $m_x(p)$, the hidden-state loss is

$$\mathcal{L}_{\text{hid}}(\theta) = \frac{1}{|\mathcal{M}|} \sum_{(s,a) \in \mathcal{M}} \mathbb{E}_{x \in \mathcal{B}} \left[\frac{\sum_p m_x(p) \left\| h_{s,p}^{(S)}(x; \theta) - h_{a,p}^{(T)}(x) \right\|_2^2}{d \sum_p m_x(p)} \right], \quad (1.31)$$

where $\mathcal{M} = \{(s, \pi(s)) : s \in \mathcal{S}(\hat{c})\}$. A useful interpretation of Equation 1.31 is that each active student layer is asked to reproduce the token representations of its assigned teacher depth. The padding mask removes padded tokens from the comparison, and the division by $d \sum_p m_x(p)$ makes the squared error comparable across sequence lengths.

Hidden-state matching gives each active layer a target representation after the layer has already combined its attention and feed-forward updates. This is useful, although it does not directly constrain how the surviving attention heads choose token interactions inside the layer. The recovery stage therefore adds pre-softmax attention supervision. This term compares the token-pair scores produced by surviving heads before those scores are converted into probabilities. Let $A_{a,r}^{(T)}(x)$ denote the pre-softmax attention score matrix of teacher anchor layer a and head r , before the row softmax is applied. Let $A_{s,r}^{(S)}(x; \theta)$ be the corresponding student score for a surviving head. With $\mathcal{H}_s$ denoting the active heads in student layer s , the attention-score loss is

$$\mathcal{L}_{\text{attn}}(\theta) = \frac{1}{|\mathcal{M}|} \sum_{(s,a) \in \mathcal{M}} \mathbb{E}_{x \in \mathcal{B}} \left[\frac{1}{|\mathcal{H}_s|} \sum_{r \in \mathcal{H}_s} \frac{\left\| P_x \left(A_{s,r}^{(S)}(x; \theta) - A_{a,\rho(r)}^{(T)}(x) \right) P_x \right\|_F^2}{\left(\sum_p m_x(p) \right)^2} \right], \quad (1.32)$$

where P_x masks padding positions and ρ maps each surviving student head to its teacher head index. A useful interpretation of Equation 1.32 is that it compares the student and teacher token-pair score tables before softmax. This is important because the softmax compresses score differences into a probability simplex. When one token already dominates a row, very different pre-softmax scores can produce similar post-softmax probabilities. Matching the scores therefore preserves more information about the attention decision than matching the probabilities alone. The denominator normalises by the number of valid token pairs, so longer sequences do not dominate the loss only because they contain more entries.

The 2C objective is

$$\mathcal{L}_{2C}(\theta; t) = \lambda_h(t) \mathcal{L}_{\text{hid}}(\theta) + \lambda_a(t) \mathcal{L}_{\text{attn}}(\theta). \quad (1.33)$$

The hidden term is the primary recovery signal. The attention term is assigned a smaller weight because it constrains the mechanism that produces the layer update. The weights may be fixed or scheduled during the stage. The classifier is frozen, so validation checkpoints measure whether the recovered body supports the existing prediction head.

For checkpoint selection, the recovery process also maintains an exponential moving average of the student parameters:

$$\bar{\theta}_t = \mu \bar{\theta}_{t-1} + (1 - \mu) \theta_t. \quad (1.34)$$

Parameter averaging reduces short-term optimisation noise and often gives a more stable validation estimate than the instantaneous weights (Polyak and Juditsky, 1992). The checkpoint selected at the end of 2C is therefore chosen from the raw or averaged parameter copy according to development accuracy.

1.8.2 2D: Prediction-Level Distillation

After 2C, the masked student has been trained toward the teacher’s internal trajectory. The classifier is then unfrozen and the objective shifts to prediction behaviour. This stage uses two signals: the ground-truth task labels and the teacher’s pre-softmax logits. The labels anchor the

model to the dataset, whereas the teacher logits provide soft class relations that are absent from one-hot labels.

Let $z^{(\mathcal{T})}(x)$ and $z^{(\mathcal{S})}(x; \theta)$ be the teacher and student logits. With label smoothing parameter ϵ , the smoothed target distribution for a class label y is

$$q_\epsilon(c \mid y) = (1 - \epsilon)\mathbf{1}[c = y] + \frac{\epsilon}{C}.$$

The task loss is

$$\mathcal{L}_{\text{ce}}(\theta) = -\mathbb{E}_{(x,y) \in \mathcal{D}_{\text{train}}} \sum_{c=1}^C q_\epsilon(c \mid y) \log \text{softmax}(z^{(\mathcal{S})}(x; \theta))_c. \quad (1.35)$$

The predictive distillation term uses temperature-scaled teacher logits:

$$\mathcal{L}_{\text{kd}}(\theta) = T^2 \mathbb{E}_{x \in \mathcal{D}_{\text{train}}} \text{KL} \left(\text{softmax} \left(\frac{z^{(\mathcal{T})}(x)}{T} \right) \parallel \text{softmax} \left(\frac{z^{(\mathcal{S})}(x; \theta)}{T} \right) \right). \quad (1.36)$$

The temperature exposes non-argmax class information, and the factor T^2 keeps the gradient scale comparable as T changes (Hinton et al., 2015). When confidence weighting is used, examples with lower teacher confidence receive a smaller KD weight.

The 2D objective is a dynamically weighted multi-objective loss:

$$\mathcal{L}_{2D}(\theta; t) = \alpha(t)\mathcal{L}_{\text{ce}}(\theta) + (1 - \alpha(t))\mathcal{L}_{\text{kd}}(\theta), \quad (1.37)$$

where $\alpha(t)$ increases slowly over the stage. Early optimisation assigns more weight to KD because the teacher distribution provides smoother gradients while the classifier adapts to the recovered representations. Later optimisation assigns more weight to CE because the final model is evaluated against task labels rather than teacher predictions.

The optimisation flow is therefore

$$\begin{aligned} \text{fixed mask} &\rightarrow 2\text{C representation recovery} \\ &\rightarrow \text{classifier unfreeze} \\ &\rightarrow 2\text{D CE/KD recovery} \\ &\rightarrow \text{best raw or EMA checkpoint.} \end{aligned} \quad (1.38)$$

The architecture remains fixed across the entire flow. Recovery trains the already masked architecture; it does not activate or deactivate heads or neurons. The recovered accuracy is therefore determined by the searched mask and the staged distillation procedure.

1.9 Experimental Protocol

This section fixes the experimental setting. The protocol keeps the task data, teacher model, search budget, recovery procedure, and reporting format constant, so differences in accuracy can be attributed to the pruning method rather than to changes in infrastructure.

1.9.1 Tasks

The experiments use two GLUE classification tasks (Wang et al., 2018). QNLI is a binary question–sentence inference task derived from SQuAD (Rajpurkar et al., 2016). MNLI is a three-way natural language inference task with matched and mismatched genres (Williams et al., 2018). We use the public training splits for search and recovery data, and the public development splits for validation. The test splits are not used. These two tasks give different stress cases: QNLI is binary and smaller, whereas MNLI is larger and has three labels.

1.9.2 Teacher and Student

The teacher is the task-fine-tuned uncased base model introduced by [Devlin et al. \(2019\)](#), built on the Transformer encoder of [Vaswani et al. \(2017\)](#). It has twelve layers, hidden dimension 768, twelve attention heads per layer, and feed forward width 3072. The student starts from the teacher weights and applies the head and neuron masks defined in Section 1.2. Search evaluates masked students without adding new modules. Recovery then trains the masked student with the distillation procedure described in Section 1.8.

1.9.3 Reproducibility Setting

Unless a variance study is explicitly reported, each run uses seed zero. When variance is reported, seeds $\{0, 1, 2\}$ are used and the table gives the mean with standard error. Runs are executed on a single GPU workstation. Mixed precision is used where stable ([Micikevicius et al., 2018](#)); the surrogate metric is evaluated in full precision because it compares small residual-update differences.

1.9.4 Search and Recovery Budget

The search uses the calibration protocol of Section 1.5 and the evolutionary procedure of Section 1.6. Candidate costs are computed with the operation model of Section 1.3, and infeasible candidates are repaired or discarded by the projection of Section 1.6.2. This keeps all search methods under the same target compute band.

After search, the selected masks are recovered with intermediate-representation distillation and prediction distillation. The recovery stage follows the distillation principle of [Hinton et al. \(2015\)](#) and the intermediate matching used in TinyBERT ([Jiao et al., 2020](#)). Optimisation uses AdamW ([Loshchilov and Hutter, 2019](#)) with warmup and cosine decay ([Loshchilov and Hutter, 2017](#)).

1.9.5 Baselines and Reporting

All baselines use the same teacher, calibration data, target budget, and recovery procedure. The comparison includes random search, beam search, and structured pruning references from CoFi ([Xia et al., 2022](#)) and ZipLM ([Kurtic et al., 2023](#)). We report development accuracy, giga operations at reference length 128, compute keep ratio, active heads, active neurons, and active layers. MNLI uses the matched development split unless stated otherwise. Differences within one standard error are treated as ties.

1.10 Conclusion

The report has described a methodology for structured pruning of a transformer encoder, applied to text classification tasks at high sparsity. The methodology combines a guided evolutionary architecture search under a hard compute budget, a surrogate distance metric scored on a small calibration set, and a multi phase distillation recovery procedure that restores task accuracy on the chosen architecture.

The conclusions the experimental evidence supports are modest. The methodology produces recovered accuracies competitive with the structured pruning literature at compute keep ratios between zero point three and zero point zero five on Question Natural Language Inference and Multi Genre Natural Language Inference. The evolutionary machinery contributes a measurable improvement over random and beam baselines. The cohort selection rule is the most consequential single design choice in the search, and the two phase structure is necessary for the recovery. The surrogate metric is a useful but imperfect predictor of recovered accuracy, with a Spearman rank correlation in absolute value around zero point five on the diagnostic set.

The methodology has known limitations that the report has documented. The surrogate’s task term saturates at high sparsity. The trajectory term has a magnitude bias. The recovery procedure has known failure modes in its exponential moving average selection and in its train versus deploy gap at half skip layers. The architectures the search produces at the highest sparsity drop layers and starve the late layers in patterns that reflect both the structure of the surrogate metric and the structure of the task.

Future work that the present analysis points to includes a per layer reweighting of the trajectory term to compensate for its magnitude bias, a brief adaptation stage before the surrogate is computed to bring it closer to a measurement of recoverability, a modification of the deployment substitution rules to preserve layer normalisations at half skip layers, and an extension of the methodology to non classification tasks. Each of these directions is testable by a controlled experiment whose protocol the report’s experimental sections provide a template for.

The methodology is, in our view, a reasonable point in the design space of structured pruning procedures. It is not a final answer. The components are individually well understood, the failure modes are documented, and the improvements remaining to be tested are concrete enough to be the subject of a follow up study.

Bibliography

- Peter Auer, Nicolò Cesa-Bianchi, Yoav Freund, and Robert E. Schapire. The nonstochastic multiarmed bandit problem. *SIAM J. Comput.*, 32(1):48–77, 2002.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019.
- Elias Frantar and Dan Alistarh. SPDY: Accurate pruning with speedup guarantees. In *ICML*, 2022.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. In *NeurIPS Deep Learning Workshop*, 2015.
- Xiaoqi Jiao et al. TinyBERT: Distilling BERT for natural language understanding. In *Findings of EMNLP*, 2020.
- Eldar Kurtic, Elias Frantar, and Dan Alistarh. ZipLM: Inference-aware structured pruning of language models. In *NeurIPS*, 2023.
- Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *ICLR*, 2017.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.
- Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? In *NeurIPS*, 2019.
- Paulius Micikevicius et al. Mixed precision training. 2018.
- Gergely Neu. Explore no more: Improved high-probability regret bounds for non-stochastic bandits. In *NeurIPS*, 2015.
- Boris T. Polyak and Anatoli B. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992.
- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *EMNLP*, 2016.
- Ashish Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *EMNLP Workshop BlackboxNLP*, 2018.
- Adina Williams, Nikita Nangia, and Samuel R. Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *NAACL-HLT*, 2018.
- Mengzhou Xia, Zexuan Zhong, and Danqi Chen. Structured pruning learns compact and accurate models. In *ACL*, 2022.